

A^TH_EN_A - Compiling

1 Introduction

This document describes how to build A^TH_EN_A from source.

The supported build target is x86_64 GNU/Linux with Qt 6 and Wayland. The reference development environment is openSUSE Tumbleweed snapshot 20260812, KDE Plasma/KWin 6.7.4 on Wayland, and Qt 6.11.1. The Tumbleweed build with Intel oneAPI icx/icpx is the configuration currently expected to compile without source changes.

Intel oneAPI C++ (icpx) is the recommended compiler. Install the current Intel oneAPI DPC++/C++ Compiler using the instructions on the [Intel compiler download and installation page](#).

2 Building A^TH_EN_A on x64 GNU/Linux

To build A^TH_EN_A for direct use, follow sections 2.1 through 2.4. To build an RPM, DEB, or AppImage, obtain the source as in 2.1 and then use 2.5.

2.1 Obtaining the source

```
git clone https://github.com/NuaptanEvalisk/ATHENA.git
cd ATHENA
```

Checkout the branch or tag that you intend to build.

2.2 Installing dependencies

Install the corresponding development packages for your distribution. On current openSUSE Tumbleweed, the following is an example:

```
sudo zypper install \
  git cmake ninja pkgconf-pkg-config patchelf \
  autoconf automake libtool make makeinfo gperf bison flex \
  qt6-base-devel qt6-svg-devel qt6-qt5compat-devel \
  qt6-wayland-devel qt6-waylandclient-devel \
  kf6-kio-devel kf6-kconfigwidgets-devel \
  kf6-kcompletion-devel kf6-syntax-highlighting-devel \
  boost-devel mimalloc-devel sqlite3-devel spdlog-devel hunspell-devel \
  freetype2-devel fontconfig-devel harfbuzz-devel \
  libpng16-devel libjpeg8-devel zlib-devel libzstd-devel \
  libsodium-devel ImageMagick-devel libgnutls-devel gmp-devel \
  libffi-devel libunistring-devel readline-devel ncurses-devel \
  libcurl-devel ghostscript resvg-devel \
  python3 python313-uv cargo rust
```

Remark 1. A system Guile installation is neither needed nor used. CMake builds the vendored A^TH_EN_A Guile 3 runtime automatically. Do not configure the build against system Guile.

Compiling libtcc

If no usable libtcc development package is available, build TinyCC from source:

```
git clone https://repo.or.cz/tinycc.git
cd tinycc
CC=icx ./configure
make -j16
```

Pass the resulting header and library to the **A^THENA** CMake configuration:

```
-DTCC_INCLUDE_DIR=/path/to/tinycc \
-DTCC_LIBRARY=/path/to/tinycc/libtcc.a
```

Compiling llama.cpp

A^THENA currently builds against the llama.cpp revision used by its container toolchain: 66c4f9ded01b29d9120255be1ed8d5835bcbb51d. A CPU build can be produced with:

```
git clone https://github.com/ggml-org/llama.cpp.git
cd llama.cpp
git checkout 66c4f9ded01b29d9120255be1ed8d5835bcbb51d
source /opt/intel/oneapi/setvars.sh
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_C_COMPILER=icx \
  -DCMAKE_CXX_COMPILER=icpx \
  -DBUILD_SHARED_LIBS=ON \
  -DLLAMA_BUILD_COMMON=ON \
  -DLLAMA_BUILD_EXAMPLES=OFF \
  -DLLAMA_BUILD_TESTS=OFF \
  -DLLAMA_BUILD_TOOLS=OFF
cmake --build build -j16
```

For an Intel SYCL-enabled build, configure llama.cpp with the additional backend options:

```
cmake -S . -B build -G Ninja \
  -DCMAKE_BUILD_TYPE=Release \
  -DCMAKE_C_COMPILER=icx \
  -DCMAKE_CXX_COMPILER=icpx \
  -DBUILD_SHARED_LIBS=ON \
  -DGGML_SYCL=ON \
  -DGGML_SYCL_TARGET=INTEL \
  -DGGML_SYCL_DNN=ON \
  -DGGML_SYCL_GRAPH=ON \
  -DLLAMA_BUILD_COMMON=ON \
  -DLLAMA_BUILD_EXAMPLES=OFF \
  -DLLAMA_BUILD_TESTS=OFF \
  -DLLAMA_BUILD_TOOLS=OFF
cmake --build build -j16
```

2.3 Compiling A^THENA

Load the oneAPI environment and choose the build parallelism. The `ATHENA_BUILD_JOBS` variable controls the private runtime builds; `-j` controls the outer CMake build.

```
source /opt/intel/oneapi/setvars.sh
JOBS=16
export ATHENA_BUILD_JOBS="$JOBS"
```

For a Debug build in build_qt6:

```
CC=icx CXX=icpx cmake -S . -B build_qt6 -G Ninja \
  -DATHENA_GUI=Qt6 \
  -DCMAKE_BUILD_TYPE=Debug \
  -DLLAMA_CPP_SOURCE_DIR=/path/to/llama.cpp
cmake --build build_qt6 -j"$JOBS"
```

For a RelWithDebInfo build in build_rel:

```
CC=icx CXX=icpx cmake -S . -B build_rel -G Ninja \
  -DATHENA_GUI=Qt6 \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DLLAMA_CPP_SOURCE_DIR=/path/to/llama.cpp
cmake --build build_rel -j"$JOBS"
```

If libtcc was built in a non-system location, add the TCC_INCLUDE_DIR and TCC_LIBRARY arguments from 2.2 to the CMake configuration.

Remark 2. The first build may spend a substantial amount of time bootstrapping the vendored Guile runtime. ThinLTO links may also take a long time.

Compiling ATHENA with ThreadSanitizer enabled

```
CC=icx CXX=icpx cmake -S . -B build_tsan -G Ninja \
  -DATHENA_GUI=Qt6 \
  -DCMAKE_BUILD_TYPE=Debug \
  -DATHENA_ENABLE_TSAN=ON \
  -DLLAMA_CPP_SOURCE_DIR=/path/to/llama.cpp
cmake --build build_tsan -j"$JOBS"
```

Remark 3. To build the test suite, configure with -DBUILD_TESTS=ON. Run it with `ctest --test-dir build_qt6 --output-on-failure` or the corresponding build directory.

2.4 Assembling the ATHENA tree

The default build stages the private Guile runtime and compiled Scheme modules into ATHENA/. Assemble or refresh the remaining runtime files with the outputs from the selected build directory:

```
BUILD=build_qt6
install -m755 "$BUILD/src/ATHENA.bin" ATHENA/bin/
install -m755 "$BUILD/src/ATHENA-Watchdog" ATHENA/bin/
install -m755 "$BUILD/src/athena-codex-bridge" ATHENA/bin/
install -m755 "$BUILD/materials-engine-cargo/release/athena-materials-engine" ATHENA/bin/
install -m755 "$BUILD/tools/athena-transmitter/athena-transmitter" ATHENA/bin/
install -m755 "$BUILD/tools/athena-web-server/athena-web-server" ATHENA/bin/
```

```
mkdir -p ATHENA/lib ATHENA/share/ATHENA/web
cp -a "$BUILD"/x64/lib/libqt6advanceddocking*.so* ATHENA/lib/
cp -a /path/to/llama.cpp/build/bin/libllama.so* ATHENA/lib/
cp -a /path/to/llama.cpp/build/bin/libggml*.so* ATHENA/lib/
cp -a tools/athena-web-server/web/. ATHENA/share/ATHENA/web/
```

Use `BUILD=build_rel` for a `RelWithDebInfo` tree. For a SYCL llama.cpp build, include the matching oneAPI runtime libraries when they are not provided by the target system.

2.5 Building RPM, DEB, and AppImage packages

The container builder requires Podman and expects Intel oneAPI under `/opt/intel/oneapi`. To build the release flavor:

```
ATHENA_BUILD_JOBS=16 ATHENA_BUILD_FLAVORS=rel \
  tools/container-build/build-athena-container.sh
```

The principal outputs are:

- `container_build/ATHENA-rel.AppImage`.
- `container_build/packages/ATHENA-VERSION-rel-linux-x86_64.deb`.
- `container_build/packages/ATHENA-VERSION-rel-linux-x86_64.rpm`.

Set `ATHENA_BUILD_FLAVORS` to `dev`, `rel`, or both as required.

3 Building A^TH_ENA on other platforms

3.1 Building A^TH_ENA for Windows

Windows is not a tested or actively maintained target. A native Windows build is theoretically possible with the necessary platform work.

The repository contains a MinGW-w64 cross-build pipeline under `tools/windows-deps`. It is currently incomplete because the vendored, modified Guile 3.0.10 runtime has not been ported to the Windows target. The pipeline must be adapted to build and package that private Guile runtime and its BDW-GC dependency before it can produce a current A^TH_ENA Windows build. The old Guile 1.8 path is obsolete.

3.2 Building A^TH_ENA for macOS, arm64 Linux, and other targets

macOS, arm64 GNU/Linux, and other platforms are theoretically possible with appropriate source, dependency, and toolchain modifications. They are not currently tested or supported build targets.

4 Getting Started with A^TH_ENA

4.1 Launching A^THENA

If you followed sections 2.1 through 2.4, start A^THENA with:

```
cd ATHENA
./StartATHENA.qt6.wayland.sh
```

To run through the watchdog:

```
./StartATHENA.qt6.wayland.sh --enable-watchdog
```

If you followed 2.1 and 2.5, install the RPM or DEB with the package manager of the target distribution, or run the AppImage directly.

Remark 4. X11/XCB support is not actively maintained and may contain defects. Native Wayland is the maintained desktop path.

Remark 5. A^THENA docking uses xdg-toplevel-drag-v1. KDE Plasma 6.1 and later support the merged protocol in KWin; GNOME 48 and later support it in Mutter. Older Wayland compositors may not support detachable-window dragging and redocking correctly. The reference system uses Plasma 6.7.4.

4.2 Setting up A^THENA Transmitter on a server

A typical deployment runs a headless A^THENA backend on loopback port 8765 and athena-transmitter in front of it on port 8766.

Generate the backend delegation keypair and start the backend:

```
ATHENA/bin/ATHENA.bin -H \
  --delegation-key-dir /var/lib/athena-backend/delegation \
  --generate-server-keypair
ATHENA/bin/ATHENA.bin -H \
  --rag-server /srv/athena/vault \
  --rag-listen-address 127.0.0.1 \
  --delegation-key-dir /var/lib/athena-backend/delegation
```

Create /etc/athena-transmitter/config.json:

```
{
  "listen_address": "127.0.0.1",
  "listen_port": 8766,
  "key_dir": "/var/lib/athena-transmitter",
  "accepted_clients": "/var/lib/athena-transmitter/accepted-clients.json",
  "pending_clients": "/var/lib/athena-transmitter/pending-clients.json",
  "upstream": {
    "url": "http://127.0.0.1:8765",
    "public_key": "BACKEND_PUBLIC_KEY",
    "fingerprint": "BACKEND_FINGERPRINT"
  }
}
```

Generate the transmitter keypair:

```
ATHENA/bin/athena-transmitter --config /etc/athena-transmitter/config.json --generate-keypair
```

Add trusted client public keys to the transmitter `accepted-clients.json`:

```
{
  "accepted": [
    "CLIENT_PUBLIC_KEY"
  ]
}
```

Add the transmitter public key to the backend `accepted-clients.json` with role proxy:

```
{
  "accepted": [
    {
      "public_key": "TRANSMITTER_PUBLIC_KEY",
      "role": "proxy"
    }
  ]
}
```

Start the transmitter:

```
ATHENA/bin/athena-transmitter --config /etc/athena-transmitter/config.json
```

Reference systemd units, the optional XClarity Controller power hook, and a Cloudflare Tunnel unit are provided under `tools/athena-transmitter/deploy`.

4.3 Setting up Web-Accessible ATHENA on a server

Build the release AppImage as in 2.5, then build the Web-Accessible ATHENA OCI image:

```
tools/athena-web-server/build-image.sh \
  --runtime container_build/ATHENA-rel.AppImage \
  --image localhost/athena-web:latest
```

Run the broker as an unprivileged user with rootless Podman and delegated cgroup v2 memory and PID controllers. On a systemd host, enable memory accounting for the broker user slice before starting it:

```
sudo systemctl set-property user-$(id -u).slice MemoryAccounting=yes
```

A basic loopback broker invocation is:

```
ATHENA/bin/athena-web-server \
  --listen-address 127.0.0.1 \
  --port 8090 \
  --image localhost/athena-web:latest \
  --max-connections 4 \
  --max-memory 4GiB \
  --storage-limit 2GiB
```

Put a WebSocket-capable TLS reverse proxy or tunnel in front of the broker. Configure STUN/TURN for the WebRTC media path as required by the deployment. The reference production templates under `tools/athena-web-server/deploy` use a dedicated openSUSE MicroOS KVM guest for the broker and rootless session containers, with Cloudflare Tunnel for signaling and Cloudflare Realtime TURN for media relay.

4.4 Next steps

Start ATHENA and read the documentation available from the Help menu. The documentation is still under construction. Additional information is available at athena.evalisk.org. For questions or build problems, contact nuaptan@evalisk.org.